

Concepto 4

Unit Testing: JUnit y Mockito.

En este documento vamos a ver lo que es Unit testing principalmente, y especialmente los tests unitarios, aquellos que buscan revisar si las clases cumplen con lo que se están pidiendo. Conceptos conectados son los de integración y de punta a punta.

Con JUnit, lo que estas usando son tres herramientas muy cercanamente acopladas, el JUnit platform, el motor que trabaja con los tests y ejecuta, hablando con el IDE y Maven y Gradle. Luego es JUnit Jupiter, la API moderna en si que agarra @Test, @BeforeEach, Assertions y conectados. Y JUnit Vintage, que provee de compatibilidad con JUnit 4 para migraciones.

Para ver esto entramos al proyecto provisto en clase encontrado en ..\testing\proyectos\01-junit-fundamentos, y hacemos correr con el comando ya muy conocido es ./mvnw test en powershell en esa carpeta. En esto vemos aplicaciones sencillas del JUnit, donde el código producción esta en Boleta.java, con la aplicación de una clase Alumno. Aquí se encuentran las reglas de negocio a probar.

En nuestro Primertest.java tenemos un unaBoletaRecienCreadaPromediaCero() , enseñando la estructura de un test de preparar el escenario, actuar el escenario que en este caso es conseguir el promedio, y por último la comprobación que con un assertEquals(0.0, promedio) nos aseguramos que el resultado realmente salga 0. Asi pasamos a los otros tests sencillos, de que la boleta al registrarse una materia con calificación el promedio regresa correcto junto con el numero de materias junto con una demostración de que DisplayName nos puede dejar mandar mensajes y no solamente estemos limitados el nombre del método. Y el tercer test nos indica que el tercer parámetro nos regresa un mensaje de fallo que podemos especificar, al mismo tiempo que probamos AssertTrue.

En el siguiente archivo que es CicludeVidaTest.java podemos ver como manejamos el ciclo de vida del testing, donde Beforeall es algo que se ejecuta antes de todos los tests una vez, beforeeach se ejecuta antes de cada test, y sus contrapartes, en donde Afterall se ejecuta después de cada test, y aftereach después de cada test. Tambien hay una explicación de porque afterall y beforeall son static, con una pequeña prueba demostrando que JUnit crea una nueva instancia por cada test.

De ahí pasamos a AsercionesTest dando mas opciones de aserciones como assertTrue, assertFalse, assertNotNull, assertNull. Nos demuestra que Assertequals no es lo mismo que assertSame. Seguimos de ahí con assertarrayequals y assertIterableEquals terminando con assertLinesMatch, con las explicaciones en código dando mas detalle sobre cada uno. De ahí vemos que con assertall fuerzas que se ejecuten todos los asserts bajo el y luego junta las que te fallaron, en contraste con las anteriores en la que, si la primera fallaba, las siguientes

no ejecutaban. Y por último vemos como un `supplier<String>` puede sustituir al `string` en el mensaje de fallo, en donde el `string` normal se construye siempre pero el `supplier` solamente si el test falla.

El siguiente archivo con `DecimalesTest.java` nos enseña como flotantes pueden ser problemáticos para hacer tests, y nos enseña el principio de deltas, o sea que tanta precisión necesitamos para nuestras reglas de negocio. En este caso al ser calificaciones, dos números decimales es suficiente.

Por último, el siguiente archivo `BoletaTest.java` nos demuestra la utilidad de ciertos tests, en donde simulamos tests completos para la clase, y al deliberadamente correr el script de `ver-fallar.sh` cambiamos una parte que falla la regla de negocio, vemos como los tests más útiles fueron los que revisaron las fronteras de valores. Nos quedamos con el consejo de que deberíamos revisar el límite, el límite más uno, el límite menos uno, vacío y nulo. También tenemos una vista previa del `assert throws`, que se ve con más detalle en el segundo proyecto. Para terminar el primer proyecto, el experimento lo que cambio fue un solo carácter en `return promedio() >= MINIMA_APROBATORIA` a `return promedio() > MINIMA_APROBATORIA`. De nuestros 22 tests, solo 2 regresaron el fallo, demostrando lo dicho con anterioridad, que los tests que si ayudan son los que revisan el límite. Puedes tener muchos tests en verde, pero solo ayudan los tests que si atrapan los errores.

Pasando a un nuevo proyecto, tenemos el `..\testing\proyectos\02-junit-catalogo`, que corremos `./mvnw test` en powershell en carpeta. Concentrándonos en la parte para demostrar `assert throws` que es lo último que nos piden demostrar, vemos que con el código de producción `Curso.java` a probar por `ExcepcionesTest.java`, que muestra la estructura de `assert throws`:

```
assertThrows(LaExcepcionQueEsperas.class, () -> elCodigoQueDebeTronar());
```

En donde la segunda parte de esto es una lambda que no se ejecuta hasta el `assertThrows` lo llama. Lo vemos demostrado por la prueba `cupoLleno()`, donde si efectivamente intentamos inscribir un tercero si nos lanza la excepción que esperamos. De ahí vemos como del `CupoLlenoException.class` podemos interrogar con `subasserts` para sacar más información en test.

Le sigue que dependiendo de que tipo de fallo estamos buscando, es más apropiado tener las excepciones adecuadas para manejarlo, usando como ejemplos el `cupollenoexception` que creamos nosotros mismos, y `IllegalStateException` y `IllegalArgumentException` para manejar bien los `assertthrows` que se nos pide. De ahí, vemos más detalles de que problema se te pueden pasar con `RuntimeException` y sus subclases, y observamos la utilidad de lo que es el `assertInstanceOf()`. Terminamos con la mención de la existencia de `assertDoesNotThrow`, útil solamente

cuando lo que quieres documentar es la ausencia de la falla, no el funcionamiento en si de la clase.

El proyecto contiene características adicionales de JUnit como `testInstance` y `testMethodOrder`, o `Nesting`, `@Tags`, `TestInfo`, `TestReporter`, `assertTimeout` y relacionados, pero como no se preguntó sobre esto específicamente se salta.

Nuestro último proyecto es el `..\testing\proyectos\04-mockito-dobles`, ejecutado con `./mvnw test` en en carpeta. Tomamos nota que nuestra clase central de producción es el `ServicioInscripcion.java` y que tiene inyector de dependencias por constructor, cosa que es importante para Mockito, que lo que hace es que no necesitamos las clases o dependencias externas y crea dobles de ellas, llevando a un ambiente mas viable de testing. Tambien es para evitarnos problemas ya que hay colaboradores que necesitamos mockear, o doblar, como por ejemplo el caso de un servicio externo lento, como `RepositorioAlumnos`, que podría hacer que nuestros tests tomen minutos en vez de segundos. También puede ser que sea difícil montar un servicio, como `RepositorioCursos`. Si queremos probar curso lleno, inscribir 30 cursos solo para probar es laborioso. Y por último tenemos el caso de `Notificacion`, que tendría efecto externo. Si quieres probar su lógica, tendrías que lidiar que su funcionamiento requiere mandar mensaje hacia el exterior, cosa que es indeseable si solo estas probando.

En `ServicioInscripcionTest.java`(que fue escogido por demostrar todo lo que se nos pide en la entrega, los otros archivos tienen evidencias más sencillas de lo siguiente pero de manera separada) demostramos el uso de Mockito, donde le decimos a JUnit con `@ExtendWith(MockitoExtension.class)` que use Mockito, creando colaboradores mock con `@Mock` de `RepositorioAlumnos`, `RepositorioCursos` y `Notificador`. Luego tenemos un `@InjectMocks` que dice que creamos un `ServicioInscripcion` real e inserto los mocks ahí.

Para demostrar `when`, en el mismo archivo de test java vemos que en la línea 79 y 80 se demuestran su uso, que en estos casos significa que cuando el servicio llama al Mock del repositorio de alumnos, que pretenda que ANA existta, lo mismo ocurre cuando busca `JAVA-101`.

Por último, terminamos con las líneas 93 y 94 que demuestran el uso de `verify`, que de manera resumida pregunta si el servicio si mando al mock apropiado una acción, en contraste a que el mock nos regrese algo con `when`. En este caso específico vemos como al preguntamos al notificador mock si se le llamo para enviar confirmación primero, al mismo tiempo que revisamos su opuesto, gracias al `never()`, que revisa que NO se envia un rechazo cuando sabemos que estamos en un test de inscribir y confirmar.

Es aquí todo lo que se nos pidió demostrar para Unit Testing con JUnit y Mockito, y solo falta constestar la pregunta, ¿qué colaborador en este pasado proyecto no deberíamos mockear? Para saber, solamente hay que ver las reglas de negocio que

estamos probando, y en este caso observamos que hay pruebas que específicamente revisan si un Curso está lleno, si tiene lugares disponibles y si se esta inscribiendo alumnos adentro de el. Si hicieras un mock de curso, le podrías decir a mockito que regrese que cuando si se pregunta si un curso esta lleno que regrese true, y en tu test estarías probando algo que ya declaraste con respuesta en vez de probar la lógica del programa. Esto, como lo dice el propio proyecto es sobremockeo.